Technical Documentation: Weather-Driven Daily Electricity Demand Forecasting for Indian States

1. Executive Overview & Architecture

This document details the complete methodology, data pipeline, preprocessing decisions, feature engineering design, and machine learning architecture implemented in and .

2. Data Acquisition & Quality Control (`src/download_data.py`)

2.1 Source & Scope

The dataset is sourced from **Zenodo Record** `14983362` (*"Daily electricity demand data for Indian states, 2014–2024"*, Hunt & Bloomfield, 2025):

- **Demand Target (`corrected_daily_energy_met_MU.csv`)**: Quality-controlled daily electrical energy met in **Million Units (`MU` , equivalent to `GWh`)** across 36 Indian states and Union Territories (`2014-01-01` to `2024-04-27` , `3,770` days).
- **Peak Demand (`corrected_peak_met_MW.csv`)**: Quality-controlled daily peak power met (`MW`).
- **State Meteorology (`weather_<State_Name>.csv` , 36 files)**: Daily population-weighted ERA5 reanalysis variables (`min` , `mean` , `max`) per state (`2014-01-01` to `2024-12-10` , `3,997` days).

2.2 Why Specific Files Were Selected vs. Skipped

- **Selected (`38 files`)**: `corrected_daily_energy_met_MU.csv` , `corrected_peak_met_MW.csv` , and all 36 `weather_<State_Name>.csv` files.
- **Excluded (`4 files`)**:
  - `daily_energy_met_MU.csv` , `peak_met_MW.csv` , `peak_unmet_MW.csv` : Raw uncorrected series containing reporting spikes and telemetry dropouts that were already cleaned in the `corrected_*` files.

- `weighted_daily_mean_utci_india.csv`: Single national aggregate series redundant with the state-level `utci` columns inside each `weather_<State>.csv` file.

3. Preprocessing Pipeline (Clean, Causal & Leak-Free)

3.1 State-Date Alignment & Unit Standardization

1. **Inner Join on Overlapping Dates**: Each state's demand series (`2014-01-01` to `2024-04-27`) is merged with its corresponding weather file on `date`. Days after `2024-04-27` (where weather exists but demand does not) are excluded.
2. **Physical Unit Conversion**:
   - Temperatures (`t2m`, `d2m`, `utci`) are converted from **Kelvin (`K`) to Celsius (`°C`)** via $T_{^\circ C} = T_K - 273.15$T∘C = TK − 273.15.
   - Horizontal 10-meter wind components (u_{10}, v_{10} u10, v10) are combined into scalar wind speed:

     $$\text{wind_speed_mean} = \sqrt{u_{10,\text{mean}}^2 + v_{10,\text{mean}}^2}$$

     wind_speed_mean = u10,mean2 + v10,mean2

3.2 Missing Value Handling (Why Causal Forward-Fill `ffill`)

- **Inspection Finding**: Across the 4 focus states (`Delhi`, `Maharashtra`, `Kerala`, `Uttar Pradesh`), weather variables are **`100%` complete (`0` missing values)**. Demand (`demand_mu`) has only **`0.40%` missing values** (`60` state-days out of `15,080`), occurring as isolated 1–2 day reporting gaps.
- **Why `ffill` Instead of Linear Interpolation**: Standard two-sided interpolation uses y_{t+1} yt+1 to fill y_t yt, introducing future data leakage. We apply **within-state forward-fill (`ffill`)** so any missing day uses only the most recent observed demand from $t-1$t − 1 (with `bfill` strictly as a fallback if day 1 of 2014 is missing).

3.3 Outlier Inspection & COVID-19 Structural Break Flag

- **Outlier Policy**: Inspection using a $3 \times \text{IQR}$3 × IQR rule flagged `0` statistical outliers in `Delhi`, `Maharashtra`, and `Kerala`, and `21` high-load summer days in `Uttar Pradesh` (`May–June 2023/2024` heatwaves). Because these represent **genuine physical heatwave peaks** that grid operators must forecast, no demand observations are clipped or deleted.
- **COVID-19 Regime Indicator (`covid_flag`)**: India's nationwide lockdowns (`March 2020 – June 2021`) depressed industrial and commercial electricity demand by `15–30%` regardless

of weather. Setting `covid_flag = 1` for `2020-03-24` through `2021-06-30` (`0` otherwise) prevents models from misattributing lockdown drops to mild spring weather.

4. Trend Normalization (`demand_index`)

4.1 Why Detrending Is Necessary

Between `2014` and `2024`, state electricity demand grew by `35–85%` due to economic expansion, rural electrification, and appliance adoption. Training a tree-based model (Random Forest or LightGBM) directly on raw `MU` causes the model to under-predict `2023–2024` test demand because tree leaves cannot extrapolate beyond the maximum `MU` observed in `2014–2021`.

4.2 Causal 365-Day Trailing Mean Formulation

For each state ss and day tt, we compute the trailing 365-day baseline using **strictly past data** shifted by 1 day ($t-365$t – 365 through $t-1$t – 1):

$$\text{trailing_365d_mean}_{s,t} = \frac{1}{365}\sum_{k=1}^{365} \text{demand_mu}_{s,t-k}$$

trailing_365d_means,t = 3651 k=1∑365 demand_mus,t–k

$$\text{demand_index}_{s,t} = \frac{\text{demand_mu}_{s,t}}{\text{trailing_365d_mean}_{s,t}}$$

demand_indexs,t = trailing_365d_means,tdemand_mus,t

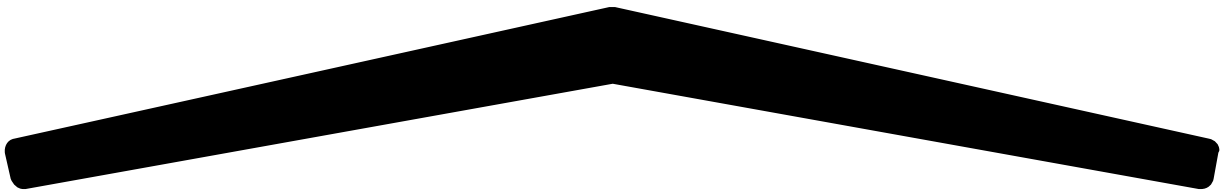
- **Interpretation**: `demand_index = 1.00` means today's load matches the state's past-year average; `1.35` means load is `35%` above the trailing annual average.
- **Exact Inverse Transformation to `MU`**: Every model predicts $\hat{\text{demand_index}}_{s,t}$ demand_index



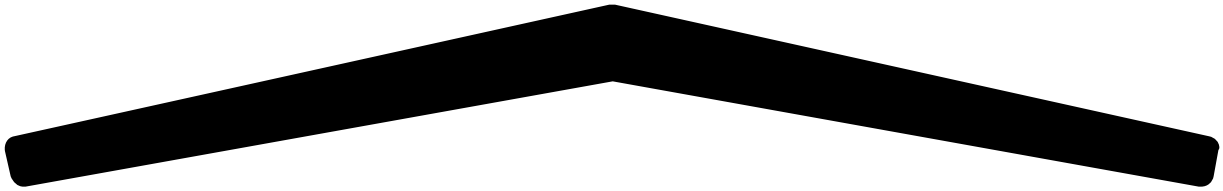
s,t, which is immediately converted back to **Million Units (`MU`)** before computing any evaluation metric:

$$\hat{\text{demand_mu}}_{s,t} = \hat{\text{demand_index}}_{s,t} \times \text{trailing_365d_mean}_{s,t}$$

demand_mu



s,t = demand_index



s,t × trailing_365d_means,t

5. Feature Engineering (`Feature Set A` vs. `Feature Set B`)

All rolling and lag features are computed **grouped by state** using past observations only. The first 7 days of 2014 (`2014-01-01` to `2014-01-07`) are dropped so `demand_index_lag7` has zero missing values.

5.1 Engineered Feature Groups

Category	Features	Physical & Operational Rationale
Calendar & Social (`6`)	`day_of_week` (`0–6`), `is_weekend` (`0/1`), `month` (`1–12`), `day_of_year` (`1–366`), `is_holiday` (state-specific via `holidays.country_holidays('IN', subdiv=...)`), `covid_flag` (`0/1`)	Captures the weekly industrial/office cycle (Sunday troughs), seasonal daylight shifts, state festival shutdowns, and lockdown anomalies.
Same-Day Weather (`7`)	`temp_mean_c` , `temp_min_c` , `temp_max_c` , `dewpoint_mean_c` , `solar_mean_jm2` , `cloud_mean_frac` , `wind_speed_mean`	Captures immediate sensible heat, night-time heat retention (`temp_min_c`), mugginess (`dewpoint_mean_c`), solar radiative load, and convective cooling (`wind_speed_mean`).
Thermal Stress (`1`)	`cooling_degree_days` $= \max(0, \text{temp_mean_c} - 22.0)$ = max(0, temp_mean_c – 22.0)	Explicitly encodes the non-linear air-conditioning activation kink above the `22°C` comfort threshold, helping linear models approximate the piecewise thermal curve.
Rolling Weather Memory (`6`)	`temp_roll7_mean` , `temp_roll30_mean` , `dewpoint_roll7_mean` , `dewpoint_roll30_mean` , `solar_roll7_mean` , `cloud_roll7_mean`	Captures **building thermal inertia** and soil moisture memory—a 5-day heatwave drives much higher AC demand on day 5 than an isolated hot day.
Autoregressive Lags (`2` , Set B only)	`demand_index_lag1` ($t-1$t – 1), `demand_index_lag7` ($t-7$t – 7)	Anchors short-term forecasts to yesterday's regime (`lag1`) and the identical weekday one week prior (`lag7`).

5.2 Two Feature Sets for Two Distinct Questions

1. **`Feature Set A` (`20 features` — Weather + Calendar Only)**:
    - *Purpose*: **Explanatory modeling & long-term climate scenario analysis**. Because it uses zero lagged demand features, it measures purely how well meteorology and calendar rules explain electricity consumption.
2. **`Feature Set B` (`22 features` — `Set A` + `demand_index_lag1` + `demand_index_lag7`)**:
    - *Purpose*: **Operational short-term forecasting**. Combines weather sensitivity with recent load persistence for day-ahead dispatch.

6. Chronological Split & Leakage Prevention

```
|<-------------------- TRAIN (8 yrs) -------------------->|<-- VAL (1 yr) -->|<--
TEST (1.3 yrs) -->|
2014-01-08                                          2021-12-31 2022-01-01  2022-12-31
2023-01-01   2024-04-27
                    2,915 days / state                      365 days / state
483 days / state
```

> IMPORTANT
>
> **Why We Never Shuffle Time-Series Data**: Randomly splitting daily records places Tuesday's weather and demand in the training set and Wednesday's in the test set. Because adjacent days share near-identical weather regimes and trailing rolling averages, random splitting leaks future state information and inflates test R^2 R2. Our chronological split guarantees that all scalers (`StandardScaler`), cross-validation splits (`TimeSeriesSplit`), and hyperparameter searches see **only `2014–2021` data**.

7. Machine Learning Models: How & Why They Were Chosen and Tuned

7.1 Model Selection Rationale

1. **Naive Baselines (`Training Mean` & `Same Weekday Last Week` y_{t-7} yt–7)**:
    - *Why*: Establishes a transparent lower bound. In electricity forecasting, `Same Weekday Last Week` (y_{t-7} yt–7) is a tough operational baseline (`Test R² = 0.760` in Delhi) because it automatically matches the season and day-of-week. Any machine learning model must beat y_{t-7} yt–7 to justify deployment.
2. **Linear Regression (Ordinary Least Squares)**:
    - *Why*: Serves as the simplest parametric model (`StandardScaler` + `LinearRegression`). Tests how much demand variance can be explained by additive linear slopes (aided by `cooling_degree_days`).
3. **Ridge Regression (ℓ_2 ℓ2-Regularized Linear Model)**:
    - *Why*: Meteorological features are strongly collinear (`temp_mean_c`, `temp_max_c`, `temp_roll7_mean`, `temp_roll30_mean` have $r > 0.90$r > 0.90). Unregularized OLS

assigns unstable, opposing weights to correlated predictors; Ridge shrinks collinear coefficients via an ℓ_2 ℓ2 penalty ($\alpha \| w \|_2^2$ α‖w‖22).

4. **Random Forest Regressor (Bagged Decision Trees)**:
    - *Why*: Captures non-linear U-shaped temperature curves (`<15°C` heating and `>24°C` cooling) and multi-way interactions (`temperature × humidity × weekend`) automatically without manual polynomial terms. Bootstrap aggregation reduces variance.
5. **LightGBM Regressor (Histogram-Based Gradient Boosted Trees)**:
    - *Why*: **Selected as the best overall model** (and the top tree family in Hunt & Bloomfield, 2025). Unlike Random Forest, which averages independent trees, LightGBM builds shallow trees sequentially where each tree corrects the residual errors of the previous ensemble—excelling at sharp seasonal transitions and peak heatwave surges.
6. **Multi-Layer Perceptron (`MLPRegressor` Neural Network)**:
    - *Why*: Mirrors the neural network baseline in Hunt & Bloomfield (2025). Uses `StandardScaler` + feed-forward hidden layers with ReLU activations (`early_stopping=True`) to learn smooth, continuous non-linear response surfaces.

7.2 Hyperparameter Tuning Protocol (`RandomizedSearchCV` + `TimeSeriesSplit`)

All tunable models are optimized on the `2014–2021` **Training Set** using a **5-fold expanding** `TimeSeriesSplit` (scoring on negative MSE of `demand_index`):

Model	Hyperparameter	Search Range	Physical / Statistical Role & Why This Range Was Chosen
Ridge	`alpha`	`[0.01, 0.1, 1.0, 10.0, 50.0, 100.0, 500.0]`	Controls ℓ_2ℓ2 shrinkage strength across 5 orders of magnitude to handle collinear temperature lags.
Random Forest	`n_estimators`	`[100, 200, 300]`	Number of bootstrap trees; `100–300` stabilizes ensemble variance without excessive runtime.
	`max_depth`	`[6, 10, 15, None]`	Caps interaction depth so trees do not memorize single days in the `2,915`-day training set.
	`min_samples_leaf`	`[2, 5, 10]`	Forces every terminal leaf to average at least `2–10` historical days, smoothing noisy outliers.
	`max_features`	`['sqrt', 0.5, 0.8]`	Subsamples features at each split so `demand_index_lag1` does not monopolize every tree root.
LightGBM	`n_estimators`	`[100, 200, 300]`	Number of boosting stages.
	`learning_rate`	`[0.03, 0.05, 0.1]`	Shrinks each tree's step size for smooth, generalizable residual fitting.
	`num_leaves`	`[15, 31, 63]`	Controls tree complexity (`15–31` leaves prevents overfitting on daily tabular series).
	`max_depth`	`[4, 6, 8, -1]`	Bounds tree depth in tandem with `num_leaves`.
	`subsample` / `colsample_bytree`	`[0.7, 0.8, 1.0]`	Row and column subsampling per tree to decorrelate boosting iterations.
	`reg_lambda`	`[0.0, 1.0, 5.0]`	ℓ_2ℓ2 penalty on leaf weights to prevent extreme predictions on unseen heatwave days.
MLP	`hidden_layer_sizes`	`[(32,), (64,), (64, 32), (128, 64)]`	Small 1–2 layer architectures suited for `~2,900` tabular rows without over-parameterization.
	`alpha`	`[0.0001, 0.001, 0.01, 0.1]`	Weight-decay (ℓ_2ℓ2) regularization on neural network weights.
	`learning_rate_init`	`[0.001, 0.005, 0.01]`	Initial step size for the Adam optimizer.

8. Comprehensive Evaluation Summary

8.1 Why LightGBM Is Selected as the Best Model

1. **Highest Accuracy on Primary Forecasting (`Set B`)**:
    - In **Delhi**, `LightGBM (Set B)` achieves the lowest Validation RMSE (`3.51 MU`, $R^2 = 0.982$R2 = 0.982) and the best out-of-sample Test performance (**`MAE = 2.25 MU`**, **`RMSE = 3.11 MU`**, $R^2 = 0.983$**R2 = 0.983**, **`MAPE = 2.47%`**), reducing error by **73%** compared to the `Same Weekday Last Week` baseline (`RMSE = 11.69 MU`).
    - Across all 4 focus states, `LightGBM (Set B)` achieves a mean Test R^2 R2 of **`0.956`** (`Delhi: 0.983`, `Uttar Pradesh: 0.965`, `Kerala: 0.956`, `Maharashtra: 0.921`).
2. **Strong Weather-Only Explanation (`Set A`)**:
    - Without any lagged demand features, `LightGBM (Set A)` explains **`94.1%`** of variance in Delhi (`RMSE = 5.78 MU`), **`87.3%`** in Kerala, and **`75.9%`** in Uttar Pradesh, outperforming Ridge Regression across every state (`+8.4%` to `+17.6%` higher R^2 R2).
3. **National Scalability (`32 States`)**:
    - When rolled out across all 32 valid Indian states/UTs, `LightGBM (Set B)` achieves a **national median Test R^2 R2 of `0.919`** and **median `MAPE` of `3.29%`**, outperforming the persistence baseline in **`32 / 32` states**.

8.2 Feature Importance Findings (Permutation Importance in `MU`)

- **Under `Feature Set A` (Weather + Calendar Only)**:
    - **Temperature dominates**: `temp_max_c` (`0.182`), `temp_mean_c` (`0.180`), `temp_roll7_mean` (`0.106`), `temp_min_c` (`0.056`), `temp_roll30_mean` (`0.050`), and `cooling_degree_days` (`0.032`) are the top meteorological predictors, followed by solar radiation (`solar_roll7_mean: 0.025`, `solar_roll30_mean: 0.020`) and humidity (`dewpoint_roll30_mean: 0.016`).
- **Under `Feature Set B` (Weather + Calendar + Lags)**:
    - `demand_index_lag1` (`0.422`) provides the baseline continuity from yesterday, while `day_of_week`, `temp_max_c`, `temp_mean_c`, `demand_index_lag7`, and `is_holiday` supply the daily weather and calendar adjustments.

9. Exported Artifacts for Phase 2 Dashboard

All Phase 2 artifacts are generated deterministically at the end of :

File Path	Size	Contents & Schema
	`3.8 MB`	Dictionary containing trained `LightGBM` pipelines (`set_a_models` and `set_b_models`) for all 4 focus states, feature column lists (`features_set_a` , `features_set_b`), best hyperparameters, and split metadata.
	`4.1 MB`	`31,264` rows covering Validation (`2022`) and Test (`2023–2024`) daily records across all 32 states, including `actual_demand_mu` , `pred_mu_set_a` , `pred_mu_set_b` , `residual_mu` , weather drivers, and `pred_high_demand_top10pct` .
	`27 KB`	Structured JSON containing all model comparison tables (`MAE` , `RMSE` , R^2 R2, `MAPE` in `MU`), 32-state rollout metrics, and Top-10% peak alert classification metrics (`Precision` , `Recall` , `F1` , confusion matrices).
	`5.9 KB`	Structured JSON of consumption insights (seasonal peak/trough months, Sunday & holiday load drops, temperature tipping points, top weather features, state weather sensitivity rankings, and grid-level recommendations).